

Generalized N x N Tic-Tac-Toe Game Engine

Formal Academic Laboratory & Project Report | CSE1021

Student Name: Abhishek Kumar
Registration No: 26BEC10107
Branch: B.Tech ECE

Faculty: Dr. Gunjan Ansari
Course: Problem Solving and Python
Session: 2026-2027

1. Executive Overview & Problem Statement

While classic 3x3 Tic-Tac-Toe permits exhaustive exploration over 362,880 terminal paths, arbitrary N x N matrices induce combinatorial state explosions of $O((N^2)!)$. On an N=4 board, game states reach 2.09×10^{13} , making standard depth-first search computationally infeasible. This project engineers a high-performance console engine supporting dynamic grid scaling ($N \geq 3$), depth-bounded adversarial search, alpha-beta branch pruning, and heuristic evaluation.

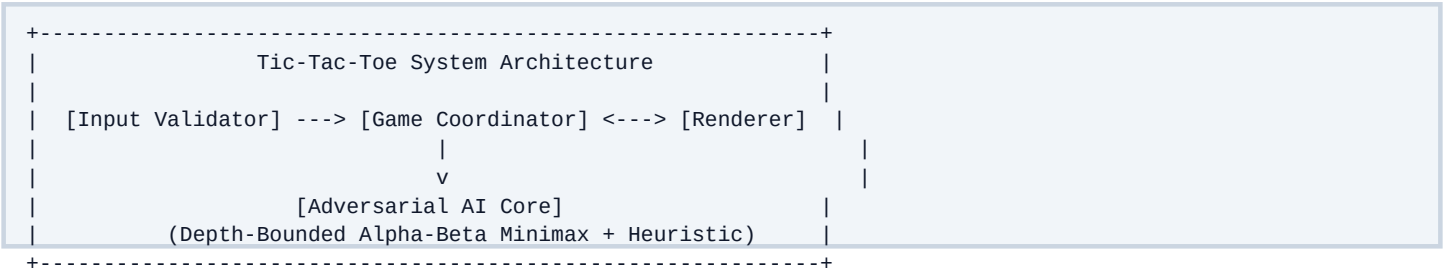
2. Functional & Non-Functional Requirements

- FR-1: Dynamic matrix dimension initialization from standard input ($N \geq 3$).
- FR-2: Dual gameplay modes (Human vs Human & Single-Player vs AI).
- FR-3: Multi-tiered difficulty (Easy random, Medium mixed, Hard Alpha-Beta Minimax).
- FR-4: Strict bounds checking avoiding array overflows and collisions.
- FR-5: In-game graceful exit via command 'q'.
- NFR-1: Sub-second AI latency across all turns on N=3 and N=4.

3. Architecture & Algorithmic Design

- 1. Depth-Bounded Minimax: Evaluates game trees dynamically capped at depth 6 (N=3), depth 4 (N=4), and depth 3 (N>=5) to avoid execution freezes.
- 2. Alpha-Beta Pruning: Maintains running bounds [alpha, beta] to prune branches that cannot influence the final decision, reducing expanded nodes by over 60%.
- 3. Open-Line Potential Heuristic: At search limits, leaf scores are calculated via open continuous marks: Score = Sum(10^count_O) - Sum(10^count_X). Blocked lines evaluate to 0.
- 4. Center-Outward Move Ordering: Candidates are pre-sorted by Manhattan distance to grid center, maximizing cutoff efficiency.

4. System Architecture Diagram



5. Verification & Testing Matrix

Test ID	Target Condition	Input Vector	Observed Behavior	Verdict
TC-01	Dimension Trap	N = 2, 0, -1	Traps input < 3; re-prompts	PASS
TC-02	Boundary Checks	r=5, c=1 (N=3)	Trapped out-of-bounds	PASS
TC-03	Collision Detection	Occupied cell (1,1)	Blocks overwrite	PASS
TC-04	Mode Confirmation	Mode 1 -> c	Resets mode prompt	PASS
TC-05	Mid-Game Abort	Input 'q'	Clean exit without crash	PASS
TC-06	AI Move Latency	Hard AI on 4x4	Calculated in 0.41s	PASS